LeetCode Practice

6 Easy: Dynamic Programming

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

6
EASY

Patterns (in order)

1. Dynamic Programming (6)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

6 questions | tap a title to jump to it

Easy questions (6) Go >

Dynamic Programming (6) Go >

E 1. Climbing Stairs LC 70

E 2. Pascal's Triangle LC 118

E 3. Best Time to Buy and Sell Stock LC 121

E 4. Fibonacci Number LC 509

E 5. Min Cost Climbing Stairs LC 746

E 6. N-th Tribonacci Number LC 1137

Easy questions (6)

EASY | TOPIC 1 OF 1

Dynamic Programming

6 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Climbing Stairs LC 70
2. Pascal's Triangle LC 118
3. Best Time to Buy and Sell Stock LC 121
4. Fibonacci Number LC 509
5. Min Cost Climbing Stairs LC 746
6. N-th Tribonacci Number LC 1137

EASY | Dynamic Programming | LeetCode #70 | 1 of 6 | 1/6

Climbing Stairs

QUESTION

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1	
Input	n = 2
Output	2
Explanation	*There are two ways to climb to the top.* *1. 1 step + 1 step* *2. 2 steps*

Example 2	
Input	n = 3
Output	3
Explanation	*There are three ways to climb to the top.* *1. 1 step + 1 step + 1 step* *2. 1 step + 2 steps* *3. 2 steps + 1 step*

CONSTRAINTS

- 1 <= n <= 45

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

You can climb 1 or 2 steps at a time. How many distinct ways to reach step n? Right?"

"A few quick questions:

n is at least 1? Yes. Order of steps matters — (1,2) and (2,1) are different? Yes."

"Let me walk: n=2 -> (1,1),(2) -> 2 ways ok. n=3 -> (1,1,1),(1,2),(2,1) -> 3 ways ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def ways(n):
    if n <= 1:
        return 1
    return ways(n - 1) + ways(n - 2)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

The same values are computed over and over. Keeping the last two values in a loop is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is recomputing the same subproblems — this is Fibonacci.

Track only the last two values — O(1) space:

a, b -> b, a+b each step.

Time: O(n). Space: O(1)."

Key insight: dp[i] = dp[i-1] + dp[i-2] is the Fibonacci recurrence — only two rolling variables are needed, cutting space from O(n) to O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _70_ClimbingStairs:
    def climbStairs(self, n):
        a, b = 1, 1
        for _ in range(n - 1):
            a, b = b, a + b
        return b
```

PHASE 5 TEST & DEBUG

"Let me trace climbStairs on n=5."

dp[0]=1 (0 steps: 1 way). dp[1]=1 (1 step: only 1). dp[2]=2 (1+1 or 2).

dp[3]=dp[2]+dp[1]=3. dp[4]=dp[3]+dp[2]=5. dp[5]=dp[4]+dp[3]=8.

Return 8 ok

WHY dp[i]=dp[i-1]+dp[i-2]: to reach step i, you came from i-1 (took 1 step) or i-2 (took 2).

This is Fibonacci! dp = [1,1,2,3,5,8,13,...].

Space optimization: only need prev two values. a=1,b=1. Loop 2..n: c=a+b; a=b; b=c. return b.

Space-optimized: a,b=1,1. for i in range(2,n+1): a,b=b,a+b. Return b.

This is exactly Fibonacci(n+1). climbStairs(n)=fib(n+1) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)
one pass	two rolling variables

Follow-up: if you can climb 1, 2, or 3 steps — same DP, track last 3 values.

Follow-up: Min Cost Climbing Stairs (#746) — add cost per step; choose min(dp[n-1],dp[n-2]).

IN YOUR APP | Expose | Editorial | Community

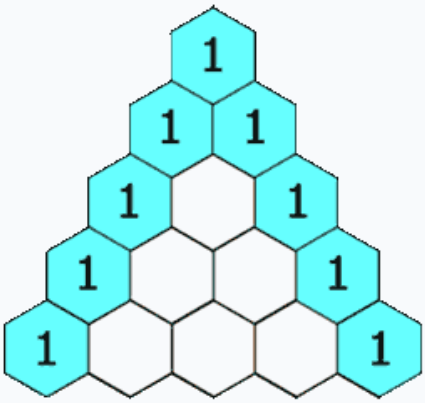
EASY | Dynamic Programming | LeetCode #118 | 2 of 6 | 2/6

Pascal's Triangle

QUESTION

Given an integer numRows, return the first numRows of Pascal's triangle.

In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1

Input	`numRows = 5`
Output	`[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]`

Example 2

Input	`numRows = 1`
Output	`[[1]]`

CONSTRAINTS

- 1 <= numRows <= 30

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Generate the first numRows rows of Pascal's triangle.

Each element = sum of two elements above it. Right?"

"A few quick questions:

Row 0 = [1]. Row 1 = [1,1]. numRows=5 -> 5 rows? Yes."

"Let me walk: numRows=5 -> [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for c in cs:
    entry = r! / (c! * (r - c)!)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^3)	O(n^2)

WHY IT IS SLOW

Each entry recomputes factorials, O(n) work for O(n^2) entries. Each entry is the sum of the two above it, O(1) each, so the whole triangle is O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Row-by-row construction — O(n²). Walk-through:"

`triangle = [[1]].`

For each row i from 1 to numRows-1:

`new_row = [1] (left edge always 1).`

For j in 1..i-1: new_row.append(triangle[i-1][j-1] + triangle[i-1][j]).

`new_row.append(1) (right edge always 1).`

`triangle.append(new_row).`

HOW each interior value is built: C(n,k) = C(n-1,k-1) + C(n-1,k) — each value is the

sum of the two values above it in the previous row.

WHY edges are 1: C(n,0) = C(n,n) = 1 by combinatorial definition.

Time: O(n²). Space: O(n²) for output."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _118_PascalsTriangle:
    def generate(self, numRows):
        triangle = []
        for i in range(numRows):
            row = [1] * (i + 1)
            for j in range(1, i):
                row[j] = triangle[i-1][j-1] + triangle[i-1
][j]
            triangle.append(row)
        return triangle
```

PHASE 5 TEST & DEBUG

"Let me trace generate(5) — Pascal's triangle."

Row 0: [1]. Row 1: [1,1]. Row 2: [1,2,1]. Row 3: [1,3,3,1]. Row 4: [1,4,6,4,1].

Each inner element row[j]=prev[j-1]+prev[j].

Row 3: [1, 1+2=3, 2+1=3, 1]. Row 4: [1, 1+3=4, 3+3=6, 3+1=4, 1].

Return [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]] ok

WHY each element is sum of two above:
C(n,k)=C(n-1,k-1)+C(n-1,k). Edges are always 1.

Edges are always 1 because C(n,0)=C(n,n)=1. Inner: C(n,k)=C(n-1,k-1)+C(n-1,k) ok

PHASE 5 TEST & DEBUG

Memory: only need previous row -> O(n) space. Row i depends only on row i-1.

dp[i][j]=dp[i-1][j-1]+dp[i-1][j]. dp[4][2]=dp[3][1]+dp[3][2]=3+3=6. dp[4]=[1,4,6,4,1] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)**
n rows, each row i has i+1 elements	for output

Follow-up: Pascal's Triangle II (#119) — return only the kth row using O(k) space.

Follow-up: binomial coefficients C(n,k) — read directly from Pascal's triangle.

EASY | Dynamic Programming | LeetCode #121 | 3 of 6 | 3/6

Best Time to Buy and Sell Stock

QUESTION

You are given an array prices where prices[i] is the price of a given stock on the ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1

Input	`prices = [7,1,5,3,6,4]`
Output	`5`
Explanation	*Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.*

Example 2

Input	`prices = [7,6,4,3,1]`
Output	`0`
Explanation	*In this case, no transactions are done and the max profit = 0.*

CONSTRAINTS

- 1 <= prices.length <= 10^5
- 0 <= prices[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Buy on one day, sell on a later day. Maximize profit. Return 0 if no profit possible. Right?"

"A few quick questions:

Must buy before sell? Yes — sell must come after buy day.

At most one transaction? Yes — buy once, sell once."

"Let me walk: prices=[7,1,5,3,6,4] -> buy at 1(day1), sell at 6(day4) -> profit=5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for day) in day)s:
    profit = prices[sell] - prices[buy]
    best = max(best, candidate)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

n(n-1)/2 pairs. One pass that tracks the lowest price so far and the best profit selling today is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the nested scan for max sell price after each buy.

Key insight: profit = sell_price - buy_price = sell_price - min_so_far.

Track min_price seen so far. At each day, profit = price - min_price.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _121_BestTimeToBuyAndSell:
    def maxProfit(self, prices):
        min_price = float('inf')
        max_profit = 0
        for price in prices:
            min_price = min(min_price, price)
            max_profit = max(max_profit, price - min_price
)
        return max_profit
```

PHASE 5 TEST & DEBUG

"Let me trace maxProfit on [7,1,5,3,6,4]."

min_price=7. max_profit=0.

Day 1: price=7. min_price=7. profit=7-7=0.

Day 2: price=1. min_price=1. profit=1-1=0.

Day 3: price=5. min_price=1. profit=5-1=4. max_profit=4.

Day 4: price=3. min_price=1. profit=3-1=2. max_profit=4.

Day 5: price=6. min_price=1. profit=6-1=5. max_profit=5.

Day 6: price=4. min_price=1. profit=3. max_profit=5.

Return 5 ok (buy at 1, sell at 6).

WHY track min_price: best buy day is the cheapest day seen so far.

WHY max_profit = max(max_profit, price-min_price): best sell day is the current day.

WHY DP terms: using state + recurrence/transition to combine prior results efficiently.

dp: min_price=7(d0)->1(d2 min). max_profit: d0->0; d1: max(0,7-7)=0; d2: max(0,1-7)=-6; d3: max(0,5-1)=4; d4: max(4,3-1)=4; d5: max(4,6-1)=5. Answer=5 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	two variables

The greedy insight: max profit = max(price - min_price_so_far) across all days.

Follow-up: Best Time II (#122) — multiple transactions; sum all positive diffs.

Follow-up: Best Time III (#123) — at most 2 transactions; 4-state DP.

IN YOUR APP | Expose | Editorial | Community

EASY | Dynamic Programming | LeetCode #509 | 4 of 6 | 4/6

Fibonacci Number

QUESTION

The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci sequence, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

```
F(0) = 0, F(1) = 1
F(n) = F(n - 1) + F(n - 2), for n > 1.
```

Given n, calculate F(n).

Example 1	
Input	n = 2
Output	1
Explanation	*F(2) = F(1) + F(0) = 1 + 0 = 1.*

Example 2	
Input	n = 3
Output	2
Explanation	*F(3) = F(2) + F(1) = 1 + 1 = 2.*

Example 3	
Input	n = 4
Output	3
Explanation	*F(4) = F(3) + F(2) = 2 + 1 = 3.*

CONSTRAINTS

- 0 <= n <= 30

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the nth Fibonacci number. F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2). Right?"

"A few quick questions:

n can be 0? Yes -> return 0. n>=0 always? Yes."

"Let me walk: F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

The call tree has about 1.6^n nodes because fib(k) is recomputed over and over. Two variables that roll forward are O(n) time and O(1) space; matrix power or fast doubling is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Iterative bottom-up DP — O(n). Walk-through:"

Base: fib(0)=0, fib(1)=1.

For i in 2..n: fib[i] = fib[i-1] + fib[i-2].

Space optimization: only keep last two values.

a, b = 0, 1.

For i in 2..n: a, b = b, a+b.

Return b.

WHY bottom-up beats naive recursion: naive recursion recomputes fib(k) for every k

multiple times -> O(2^n). Bottom-up computes each once -> O(n).

Memoized recursion also O(n) but O(n) stack space vs O(1) iterative.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _509_FibonacciNumber:
    def fib(self, n):
        if n <= 1:
            return n
        prev2, prev1 = 0, 1
        for _ in range(2, n + 1):
            prev2, prev1 = prev1, prev2 + prev1
        return prev1
```

PHASE 5 TEST & DEBUG

"Let me trace fib(5)."

dp[0]=0, dp[1]=1, dp[2]=1, dp[3]=2, dp[4]=3, dp[5]=5 ok

Space-optimized: prev=0, curr=1

n=2: prev,curr=1,1; n=3: prev,curr=1,2; n=4: prev,curr=2,3; n=5: prev,curr=3,5

Return curr=5 ok

WHY two variables: fib(n) only needs fib(n-1) and fib(n-2); no need for full array.

Recursive: fib(5)->fib(4)+fib(3)->... exponential without memoization.

Edge: fib(0)=0, fib(1)=1 (base cases).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Matrix exponentiation achieves O(log n) — useful for very large n.

Follow-up: Climbing Stairs (#70) — same Fibonacci recurrence.

Follow-up: N-th Tribonacci Number (#1137) — three rolling variables.

IN YOUR APP | Expose | Editorial | Community

EASY | Dynamic Programming | LeetCode #746 | 5 of 6 | 5/6

Min Cost Climbing Stairs

QUESTION

You are given an integer array cost where cost[i] is the cost of ith step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1

Input	`cost = [10, 15 ,20]`
Output	`15`
Explanation	*You will start at index 1. - Pay 15 and climb two steps to reach the top. The total cost is 15.*

Example 2

Input	`cost = [ 1 ,100, 1 ,1, 1 ,100, 1 , 1 ,100, 1 ]`
Output	`6`
Explanation	*You will start at index 0. - Pay 1 and climb two steps to reach index 2. - Pay 1 and climb two steps to reach index 4. - Pay 1 and climb two steps to reach index 6. - Pay 1 and climb one step to reach index 7. - Pay 1 and climb two steps to reach index 9. - Pay 1 and climb one step to reach the top. The total cost is 6.*

CONSTRAINTS

- 2 <= cost.length <= 1000
- 0 <= cost[i] <= 999

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each step has a cost; you can start at index 0 or 1, and move 1 or 2 steps at a time.

Find the minimum cost to reach the top (just past the last index). Right?"

"A few quick questions:

Paying cost[i] happens when you STEP ON i, and the top has no cost (you just need to reach past it)? Yes."

"Let me walk: cost=[10,15,20] -> start at 1(pay15), step 2 to top -> 15 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= n:
        return 0
    return cost[i] + min(best(i + 1), best(i + 2))
    answer = min(best(0), best(1))
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches per stair and the same stair repeats. Keeping the cost to reach the last two stairs in two variables is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Bottom-up DP — O(n). Walk-through:"

`dp[i] = minimum cost to reach step i.`

`dp[0] = cost[0]. dp[1] = cost[1].`

For i in 2..n:

`dp[i] = cost[i] + min(dp[i-1], dp[i-2])`

<- pay cost[i] to stand on step i, then jump 1 or 2 steps.

WHY dp[n] (one step past end): the 'top' is one step beyond the last cost entry.

`dp[n] = min(dp[n-1], dp[n-2]) (top itself has no cost).`

Return dp[n] (or equivalently: min(dp[n-1], dp[n-2]) without the extra entry).

Time: O(n). Space: O(1) with two variables.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _746_MinCostClimbingStairs:
    def minCostClimbingStairs(self, cost):
        prev2, prev1 = 0, 0
        for i in range(2, len(cost) + 1):
            current = min(prev1 + cost[i-1], prev2 + cost[
i-2])
            prev2, prev1 = prev1, current
        return prev1
```

PHASE 5 TEST & DEBUG

"Let me trace minCostClimbingStairs([10,15,20])."

`dp[i] = min cost to reach step i (beyond array = top)`

dp[0]=10 (cost to leave step 0), dp[1]=15 (cost to leave step 1)

dp[2]=min(dp[0],dp[1])+cost[2]=min(10,15)+20=30

Answer = min(dp[1],dp[2])=min(15,30)=15 ok

(Take step 1 for cost 15, jump 2 to top)

Key insight: dp[i]=min(dp[i-1]+cost[i-1],dp[i-2]+cost[i-2]) — optimal substructure via min of two predecessors.

Edge: cost=[1,100,1,1,100,1,1,1]: alternating cheap/expensive; dp gives optimal path.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

Follow-up: Climbing Stairs (#70) — same recurrence without costs, just count ways.

Follow-up: House Robber (#198) — similar adjacent-choice DP pattern.

EASY | Dynamic Programming | LeetCode #1137 | 6 of 6 | 6/6

N-th Tribonacci Number

QUESTION

The Tribonacci sequence T_n is defined as follows:

$T_0 = 0$, $T_1 = 1$, $T_2 = 1$, and $T_{n+3} = T_n + T_{n+1} + T_{n+2}$ for n >= 0.

Given n, return the value of T_n.

Example 1	
Input	n = 4
Output	4
Explanation	*T_3 = 0 + 1 + 1 = 2* *T_4 = 1 + 1 + 2 = 4*

Example 2	
Input	n = 25
Output	1389537

CONSTRAINTS

- 0 <= n <= 37
- The answer is guaranteed to fit within a 32-bit integer, ie. answer <= 2^31 - 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

T(0)=0, T(1)=1, T(2)=1, T(n)=T(n-1)+T(n-2)+T(n-3) for n>=3. Return T(n). Right?"

"A few quick questions:

n can be 0,1,2 — just return the base cases directly? Yes."

"Let me walk: n=4 -> T(3)=0+1+1=2, T(4)=1+1+2=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def trib(n):
    if n < 3:
        return the base value
    return trib(n - 1) + trib(n - 2) + trib(n - 3)
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three recursive calls per step, recomputing the same values. Three rolling variables are O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Bottom-up DP with three variables — O(n). Walk-through:"

T(0)=0, T(1)=1, T(2)=1. For n<3: return directly.

a, b, c = 0, 1, 1 (T(0), T(1), T(2)).

For i in 3..n: a, b, c = b, c, a+b+c.

Return c.

WHY three variables: T(n) = T(n-1) + T(n-2) + T(n-3) — need three previous values.

Rolling update: shift a<-b, b<-c, c<-a+b+c avoids an array.

Same pattern as Fibonacci but with three terms instead of two.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1137_NthTribonacciNumber:
    def tribonacci(self, n):
        if n == 0:
            return 0
        if n <= 2:
            return 1
        t0, t1, t2 = 0, 1, 1
        for _ in range(3, n + 1):
            t0, t1, t2 = t1, t2, t0 + t1 + t2
        return t2
```

PHASE 5 TEST & DEBUG

"Let me trace tribonacci(4)."

T(0)=0, T(1)=1, T(2)=1

T(3)=T(0)+T(1)+T(2)=0+1+1=2

T(4)=T(1)+T(2)+T(3)=1+1+2=4 ok

Space-optimized: a,b,c=0,1,1

n=3: a,b,c=1,1,2; n=4: a,b,c=1,2,4 -> return 4 ok

WHY rolling 3 vars: tribonacci only needs 3 previous; no array needed.

Edge: n=0->0, n=1->1, n=2->1 (direct base cases).

dp[i]=T(i). dp[0]=0,dp[1]=1,dp[2]=1. dp[i]=dp[i-1]+dp[i-2]+dp[i-3]. dp[3]=2,dp[4]=4 ok. Optimized: rolling 3-var window replaces dp array but dp[i] recurrence is the core.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: Fibonacci Number (#509) — same rolling-variable pattern with 2 instead of 3 values.

Follow-up: Climbing Stairs (#70) — Fibonacci recurrence in disguise.